

Guia de Apresentação do Backend

Portal SENA — Roteiro de Estudo e Explicação para a Apresentação

Este guia prático foi elaborado para orientá-lo durante a apresentação do backend para o professor. Cada um dos 12 tópicos está mapeado diretamente para o arquivo e trecho de código mais legíveis no seu projeto, acompanhado de uma sugestão de fala teórica/prática.

Visão Geral da Arquitetura do Sistema

O backend foi desenvolvido em **Java** seguindo o padrão arquitetural **MVC (Model-View-Controller)** integrado com a camada **DAO (Data Access Object)** para acesso ao banco PostgreSQL hospedado no Supabase:

- **Model:** Representa os dados e as regras de negócio (`Usuario` , `Imovel` , `Avaliacao` , `TipoConta`).
- **Controller:** Controla as rotas e requisições HTTP (`LoginHandler` , `CadastroHandler` , etc.).
- **Database:** Gerencia a conexão com o banco e executa as queries SQL (`DatabaseConnection` , `UsuarioDAO` , `ImovelDAO`).
- **GUI:** Painel administrativo opcional em Swing para visualizar os logs e gerenciar o servidor HTTP.

Os 12 Tópicos Mapeados no Código

1. Classes e Objetos

[backend/model/Imovel.java](#)

Conceito: A Classe é o projeto/molde de uma entidade, enquanto o Objeto é uma instância física e viva daquela classe contendo dados reais na memória.

"Professor, no nosso projeto, a classe **Imovel** define os atributos de qualquer imóvel no sistema (id, titulo, preco, etc.). Criamos instâncias físicas (objetos) dessa classe em memória, por exemplo, no **CadastroImovelHandler** quando recebemos dados da web e fazemos `new Imovel(...)` ."

2. Construtores

[backend/model/Usuario.java](#) e [Administrador.java](#)

Conceito: Método especial executado no momento da instanciação de um objeto para inicializar seu estado interno e seus atributos.

"Nós usamos construtores para inicializar as propriedades dos nossos objetos. Na classe base **Usuario**, o construtor recebe *id*, *nome*, *email*, *senha* e *tipo de conta*. Em subclasses como **Administrador**, usamos a palavra-chave `super(...)` para delegar essa inicialização para o construtor pai."

3. Encapsulamento

[backend/model/Usuario.java](#)

Conceito: Mecanismo que restringe o acesso direto aos detalhes internos de um objeto, protegendo-os com o modificador `private` e expondo métodos de acesso controlados (Getters e Setters).

"Para garantir a segurança dos dados, todos os atributos da classe **Usuario** foram declarados como `private`. O acesso externo a eles é restrito aos métodos públicos correspondentes, como `getNome()` e `setNome(String nome)`."

4. Herança

[backend/model/Administrador.java](#), [Anfitriao.java](#), [Hospede.java](#)

Conceito: Mecanismo que permite a uma classe herdar propriedades e comportamentos de outra classe, promovendo o reuso e a especialização.

"Temos três tipos de contas no Portal SENA. Em vez de duplicar campos de login em cada uma, criamos a classe base **Usuario**, e as classes **Administrador**, **Anfitriao** e **Hospede** herdam dela usando a palavra-chave `extends`."

5. Interfaces

[backend/database/DAO.java](#)

Conceito: Contrato que define um conjunto de assinaturas de métodos (sem corpo) que as classes implementadoras são obrigadas a programar.

"Nós criamos a interface genérica `DAO<T>` para padronizar nossos repositórios de banco de dados. Ela força qualquer classe DAO do sistema a implementar os métodos de CRUD (`getAll`, `getById`, `save`, `update`, `delete`)."

6. Classes Abstratas

[backend/model/Usuario.java](#) e [AbstractHttpHandler.java](#)

Conceito: Classe conceitual que serve apenas como molde para outras e não pode ser instanciada diretamente com `new`.

"A classe **Usuario** é abstrata (`public abstract class Usuario`). Isso impede que criemos um usuário genérico no sistema. Forçamos a criação de perfis reais (subclasses) e compartilhamos a lógica comum do protocolo HTTP na classe **AbstractHttpHandler**."

7. Polimorfismo

[backend/database/UsuarioDAO.java](#) e [LoginHandler.java](#)

Conceito: Capacidade de um objeto ser referenciado de várias formas. Pode ocorrer por Sobrescrita de métodos ou Polimorfismo de Referência.

"No método `mapearUsuario` em **UsuarioDAO**, o retorno declarado é `Usuario` (polimorfismo de referência), mas criamos dinamicamente instâncias reais de `Hospede`, `Anfitriao` ou `Administrador` dependendo do banco de dados. Além disso, aplicamos sobrescrita de método (`@Override`) nos manipuladores HTTP."

8. Coleções do Java

[backend/model/Imovel.java](#)

Conceito: Estruturas de dados dinâmicas para armazenar e manipular grupos de objetos de forma eficiente.

"Para agrupar as avaliações recebidas por um imóvel de forma dinâmica, declaramos `private List<Avaliacao> avaliacoes = new ArrayList<>()` . Isso nos dá a flexibilidade de adicionar avaliações em tempo de execução sem limite fixo de tamanho."

9. Java IO

[backend/database/DatabaseConnection.java](#) e [ServerConsoleFrame.java](#)

Conceito: Fluxos de entrada e saída (Streams) de dados para interagir com recursos externos, como arquivos ou fluxo padrão de saída.

"Nós usamos `InputStream` em **DatabaseConnection** para ler de forma segura o arquivo `db.properties` . E em **ServerConsoleFrame**, criamos um `OutputStream` personalizado para capturar as mensagens do console Java e exibi-las na tela de logs do servidor."

10. Enum em Java

[backend/model/TipoConta.java](#)

Conceito: Tipo especial de dados em Java que permite que uma variável seja um conjunto de constantes pré-definidas.

"Criamos o enum **TipoConta** com os valores fixos `HOSPEDE` , `ANFITRIAIO` e `ADMINISTRADOR` . Ele encapsula uma descrição textual amigável e possui um método utilitário estático para fazer a conversão a partir de textos do banco de dados."

11. Exceções do Java

[backend/exception/SenaException.java](#) e [UsuarioDAO.java](#)

Conceito: Tratamento robusto de erros em tempo de execução usando `try-catch-resources` e a criação de exceções personalizadas herdadas de `Exception`.

*"Nós criamos nossa própria hierarquia de exceções de negócio com **SenaException** e **DatabaseException**. Além disso, em nossos DAOs, usamos o **Try-With-Resources** (como em `try (Connection conn = ...)`), que gerencia a liberação automática de conexões com o banco, evitando vazamento de memória."*

12. Thread

[backend/gui/ServerConsoleFrame.java](#)

Conceito: Execução de linhas de processamento concorrentes de forma assíncrona.

*"Para evitar que a tela Swing trave, nós rodamos o servidor HTTP e o teste de conexão do Supabase em **Threads** separadas em segundo plano. Desse modo, o Swing Event Dispatch Thread permanece livre para responder às interações do usuário."*



Dicas Cruciais para a Apresentação

1. Comece pelo painel visual

Mostre o painel administrativo Swing rodando. Clique em "**Testar Conexão Supabase**" para mostrar as logs aparecendo em tempo real. Isso prova que a aplicação está viva e integrada ao banco de dados em nuvem.

2. Foco no Try-With-Resources

Explique ao professor que ao declarar a conexão do banco de dados dentro do cabeçalho do `try`, o próprio Java se encarrega de fechar tudo ao término do bloco, dispensando o antigo e verboso bloco `finally`.

3. Por que usar Threads na interface gráfica?

Se o professor perguntar o motivo de usar Threads separadas, lembre-se: *"Tarefas de I/O de rede (servidor web e consultas remotas) são bloqueantes. Se executadas na thread principal da GUI, a tela congela e para de responder a cliques."*